From Paper to Proof

Session 4: Your own theorem

Michail Karatarakis

19 November 2026

BGSMath Graduate Course · UPC, Barcelona

The course

	Session	Content
1	Lean and Mathlib	What formalizing means, and does not mean; first proofs; finding things in Mathlib; reading a Lean statement
2	From paper to Lean	Writing definitions and statements; the usual mistakes; drafting statements with AI and checking them
3	AI tools	What AI did this year; an agent proving a statement, live; checking what it hands back
4	Your own theorem	Statements sent in by participants, formalized live; how to do your own at home

Today: one theorem from start to finish, then some of yours, live.

The course in one slide

Read	Brackets, junk values, coercions, quantifier order. The checklist. (Session 1)
Write	Definitions with tests. Statements whose hypotheses can hold, and are all needed. (Session 2)
Prove	An agent writes the proof against a pinned statement. (Session 3)
Check	No `sorry`, the three standard axioms, the statement unchanged. (Session 3)

> Lean checks the **proof**. You check the **statement**.

Today

1. **One theorem, start to finish.** Everything from the course, on one small example.
2. **Your theorems, live.** Most of the session: two or three of the statements you sent, with the same steps.
3. **At home:** your own theorem, from `MyTheorem.lean`.

One theorem, start to finish

The statement

Theorem

Every continuous $f : [0,1] \to [0,1]$ has a fixed point.

- "$f : [0,1] \to [0,1]$" becomes two hypotheses: continuity on $[0,1]$, and *maps into* $[0,1]$.
- Pinned in a `def`, as in session 3.

In Lean

```
-- Paper: a continuous f : [0, 1] → [0, 1]
-- has a fixed point.
def Statement : Prop :=
  ∀ f : ℝ → ℝ,
    ContinuousOn f (Icc 0 1) →
    (∀ x ∈ Icc (0 : ℝ) 1, f x ∈ Icc 0 1) →
    ∃ c ∈ Icc (0 : ℝ) 1, f c = c
```

Test it before proving it

live in Lean

```
-- The hypotheses can hold: f = 1/2.
example : ContinuousOn (fun _ : ℝ => (1 / 2 : ℝ))
      (Icc 0 1) ∧
    ∀ x ∈ Icc (0 : ℝ) 1, (1 / 2 : ℝ) ∈ Icc 0 1 :=
  ⟨continuousOn_const, fun _ _ => by norm_num⟩

-- Without "f maps into [0, 1]": f x = x + 1.
example : ¬ ∀ f : ℝ → ℝ, ContinuousOn f (Icc 0 1) →
    ∃ c ∈ Icc (0 : ℝ) 1, f c = c := by
  intro h
  obtain ⟨c, -, hc⟩ := h (fun x => x + 1) (by fun_prop)
  linarith
```

- The hypotheses can hold together: a constant function.
- The “maps into” hypothesis is needed: without it, $x \mapsto x + 1$ is a counterexample.

The plan, on paper

Proof

Let $g(x) = f(x) - x$. Then $g(0) = f(0) \geq 0$ and $g(1) = f(1) - 1 \leq 0$. By the intermediate value theorem there is $c \in [0, 1]$ with $g(c) = 0$, that is, $f(c) = c$. □

- One step needs another theorem: the intermediate value theorem.
- State it precisely first, in the form the proof needs. Then look for it.

A conditional proof

```
-- The step we need, stated precisely.
def IVT : Prop :=
  ∀ g : ℝ → ℝ, ContinuousOn g (Icc 0 1) →
    g 1 ≤ 0 → 0 ≤ g 0 →
    ∃ c ∈ Icc (0 : ℝ) 1, g c = 0

theorem main_of_ivt (h : IVT) :
    Statement := by
  intro f hf hmaps
  have h0 := (hmaps 0 ⟨le_rfl, zero_le_one⟩).1
  have h1 := (hmaps 1 ⟨zero_le_one, le_rfl⟩).2
  obtain ⟨c, hc, hgc⟩ :=
    h (fun x => f x - x)
      (hf.sub continuousOn_id)
      (by linarith) (by linarith)
  exact ⟨c, hc, by linarith⟩
```

- The missing step is a **hypothesis** of the theorem, not a `sorry` inside it.
- Anyone reading the statement sees exactly what is assumed.
- The same shape as the conditional formalization of the unit distance disproof (session 3).

Closing the gap

```
theorem ivt : IVT := by
  intro g hg h1 h0
  exact intermediate_value_Icc' zero_le_one hg
    ⟨h1, h0⟩

theorem main : Statement :=
  main_of_ivt ivt
```

- Mathlib has the intermediate value theorem: `intermediate_value_Icc'`.
- If it did not, `main_of_ivt` would still be an honest result, with its assumption in plain view.

The audit

live in Lean

```
#print axioms main_of_ivt
-- depends on axioms:
--   [propext, Classical.choice,
    Quot.sound]
#print axioms main
-- depends on axioms:
--   [propext, Classical.choice,
    Quot.sound]
```

- `Statement` is unchanged.
- No `sorry`; only the three standard axioms.
- The tests passed before the proof was written.

> Statement, tests, plan, proof, audit. The same steps work for your theorem.

Your theorems, live

Your theorems

live in Lean

For each statement you sent:

1. The paper statement, with every hypothesis, including the implicit ones.
2. The Lean statement, drafted with an AI. Read it aloud, as in session 1.
3. The tests: the hypotheses can hold, and each one is needed.
4. The proof: an agent, against the pinned statement.
5. The audit.

> The question for each one: does the Lean statement say what the paper says? You decide.

At home

Your own theorem

Open `MyTheorem.lean`. It has one section per step:

1	Paper statement	Every hypothesis, including the implicit ones.
2	Definitions	Only if Mathlib has none. Test each one.
3	Statement	Pinned. The agent may not edit it.
4	Tests	Hypotheses can hold; each one is needed; a case you know.
5	Steps	The plan, as precise statements.
6	Proof	You, an agent, or both.
7	Audit	`#print axioms main`.

Steps 1–4 matter most. A well-tested statement with an open proof is a good outcome.

When you are stuck

- **A definition is missing.** Search Mathlib first (Loogle, LeanSearch, the documentation). If it really is missing, write it and test it (session 2).
- **A step will not go through.** State it as its own lemma, and give the agent only that lemma.
- **The agent keeps failing.** Ask it to try the negation, or to look for a counterexample. The statement may be false.
- **The gap is real.** Make it a hypothesis, as in `main_of_ivt`. A conditional result with a precise assumption is progress.
- **You are lost.** Read the infoview, then ask on the Lean Zulip.

After the course

Where to go next

- **The Lean Zulip chat**, stream *#new members*: questions of every level are welcome.
- **Mathematics in Lean**, for the parts of Mathlib you did not see here.
- **Contributing to Mathlib**: a result in the right generality, with Mathlib's names and style, reviewed by maintainers. Start with something small.
- **Larger projects**: Tau Ceti roadmaps, Prove2Me missions, or a blueprint of your own.

The one idea of the course

> Lean checks the **proof**. You check the **statement**.

- AI now writes a large part of the proofs. That makes the statement the part that needs you.
- Read it, test it, pin it, audit what comes back.

Thank you.